

Template Method Design Pattern (Behavioral)

Define the skeleton of an algorithm once in a base class, and let subclasses supply the varying steps without ever changing the algorithm's structure or the order its steps run in.

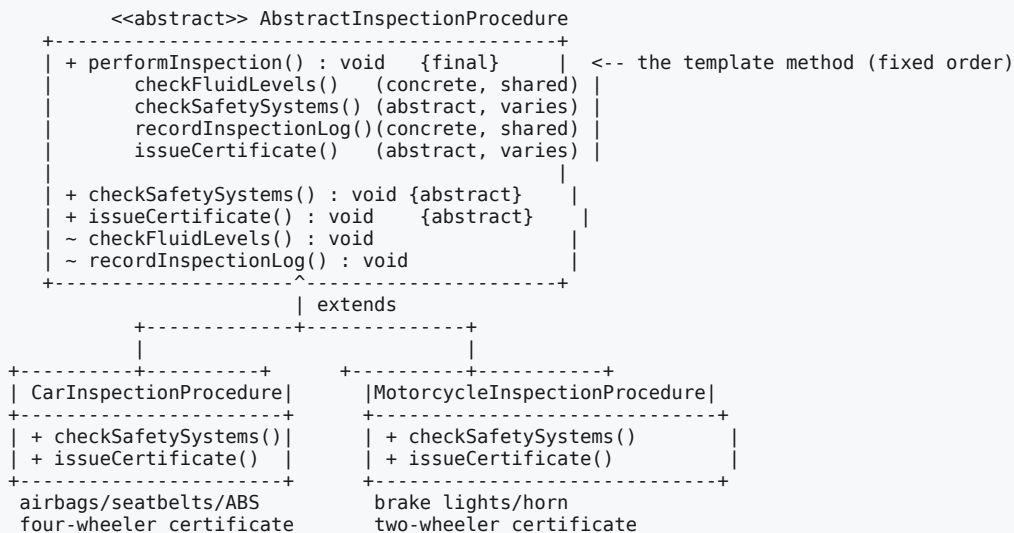
Intent

The Template Method pattern solves the problem of *duplicated procedures that differ only in a few steps*. Several classes may follow the exact same overall procedure — the same steps, in the same order — while differing in how one or two of those individual steps behave. If each class wrote the whole procedure itself, the shared structure would be copied into every class, and any copy could drift: a step could be reordered, skipped, or forgotten.

Template Method fixes this by placing the invariant part — the sequence and control flow — in a single method on an abstract base class (the *template method*), and expressing the variable parts as separate step methods. Steps that never change are implemented once in the base class; steps that vary are declared `abstract` and deferred to subclasses. The result is that the algorithm's shape is written exactly once and reused, while each subclass customizes only what genuinely differs.

Here the algorithm is **inspecting a vehicle at a roadworthiness testing station**: check fluid levels, check the safety systems, record the inspection log, issue a certificate. That order is fixed for every vehicle that rolls through the station. Only two steps differ between vehicle classes — *which safety systems get tested* (airbags/seatbelts/ABS for a car vs. brake lights/horn for a motorcycle) and *which certificate gets issued* (a four-wheeler certificate vs. a two-wheeler certificate). Those two become the customization points; everything else is shared.

UML class diagram (ASCII)



`performInspection()` is the template method — it is `final`, so it is drawn as owned by the base class and cannot be redefined. The two `abstract` methods are the "holes" each subclass fills; the two concrete methods (`~` = package-private) are the shared steps subclasses inherit unchanged.

The players

- AbstractInspectionProcedure** — The abstract base class. It contains the *entire pattern*: the `final` template method `performInspection()` that defines the fixed step order, two `abstract` primitive operations (`checkSafetySystems()`, `issueCertificate()`) that subclasses must implement, and two concrete invariant steps (`checkFluidLevels()`, `recordInspectionLog()`) implemented once and shared with every subclass.
- CarInspectionProcedure** — A concrete subclass. Extends `AbstractInspectionProcedure` and overrides only the two varying steps to inspect a car (test airbags/seatbelts/ABS, issue a four-wheeler certificate). It contributes nothing to the algorithm's structure.
- MotorcycleInspectionProcedure** — The other concrete subclass. Same shape as `CarInspectionProcedure`, but tests brake lights and horn, and issues a two-wheeler certificate. It runs the byte-for-byte same skeleton `CarInspectionProcedure` runs.

- **VehicleInspectionStation** — The client / driver class. Contains `main()`. It creates a `CarInspectionProcedure` and a `MotorcycleInspectionProcedure`, holds each through the base type `AbstractInspectionProcedure`, and calls `performInspection()` on each to demonstrate that one shared algorithm produces two different inspection runs.

Code walkthrough — every line explained

AbstractInspectionProcedure.java

```
package com.design.patterns.templateMethod.template;

public abstract class AbstractInspectionProcedure {

    public final void performInspection() {
        checkFluidLevels();
        checkSafetySystems();
        recordInspectionLog();
        issueCertificate();
    }

    public abstract void checkSafetySystems();

    public abstract void issueCertificate();

    void checkFluidLevels() {
        System.out.println("Checking engine oil and coolant levels");
    }

    void recordInspectionLog() {
        System.out.println("Recording inspection log entry");
    }
}
```

Line by line:

- `package com.design.patterns.templateMethod.template;` — Declares that this class belongs to the `...templateMethod.template` package. Packages organize related classes into a namespace and prevent name collisions. Putting the base class and its shared steps in a `template` sub-package cleanly separates the reusable skeleton from the concrete inspection procedures, which live in the `template.concrete` sub-package below it.
- `public abstract class AbstractInspectionProcedure {` — Declares a `public` top-level class named `AbstractInspectionProcedure`, marked `abstract`. `public` means any package may reference it. `abstract` means the class cannot itself be instantiated (`new AbstractInspectionProcedure()` is a compile error) and may declare `abstract` methods with no body. This is essential: the class is intentionally incomplete — it knows the *inspection routine* but not how to *test safety systems* or *issue a certificate*, so it must not be instantiable on its own. The `{` opens the class body.
- `public final void performInspection() {` — Declares the **template method**, the heart of the pattern. `public` makes it the callable entry point clients use. `final` forbids any subclass from overriding it — this locks the algorithm's structure so no subclass can change the routine (see *Why these design decisions*). `void` because performing an inspection produces console output, not a return value. The `{` opens the method body.
- `checkFluidLevels();` — The first step of the algorithm. It calls the concrete `checkFluidLevels()` method defined lower in this same class. Because it is a shared, invariant step, every vehicle runs the identical implementation here. This line fixes "check fluid levels" as step 1.
- `checkSafetySystems();` — The second step. This is a call to the `abstract` method `checkSafetySystems()`, which has no body in this class. At runtime, Java's dynamic dispatch (polymorphism) routes this call to the actual subclass's override — `CarInspectionProcedure.checkSafetySystems()` or `MotorcycleInspectionProcedure.checkSafetySystems()` — depending on the real object type. The base class thus calls *down* into subclass code at exactly the right moment without knowing which subclass it is.
- `recordInspectionLog();` — The third step, calling the shared concrete `recordInspectionLog()`. Fixed for every vehicle, so it is implemented once and reused. This line pins "record inspection log" as step 3, always *after* the safety check and *before* the certificate is issued.
- `issueCertificate();` — The fourth and final step, another `abstract` call dispatched to the subclass's override. This is the second customization point.
- `}` — Closes `performInspection()`. The four lines above, in this exact sequence, are the invariant algorithm. Because the method is `final`, this order is guaranteed for every subclass, forever.
- `public abstract void checkSafetySystems();` — Declares `checkSafetySystems` as an `abstract` method: a signature with no body, ending in a semicolon. `abstract` forces every concrete subclass to provide its own implementation — you cannot compile a non-abstract subclass that forgets it; the compiler enforces the contract. `public` matches the visibility the template method needs. This is a *primitive operation*: a step the base class deliberately refuses to define because it genuinely varies per vehicle class.

- `public abstract void issueCertificate();` — The second `abstract` primitive operation, declared for the same reason. Each subclass must decide which certificate to issue. Declaring it `abstract` both *requires* the subclass to supply it and *documents* that this is one of the two intended customization points.
- `void checkFluidLevels() {` — Declares a concrete (has a body) *invariant step*. It has no access modifier, so it is **package-private** — visible only within the `...template` package. That is enough because the only caller is `performInspection()` in this same class; the method is an internal step, not part of the public customization surface subclasses are meant to override. The `{` opens its body.
- `System.out.println("Checking engine oil and coolant levels");` — Performs the actual work of the step: writes the line `Checking engine oil and coolant levels` to standard output, followed by a newline. In a real inspection station this would involve a physical fluid check; here it prints a trace line so the fixed algorithm is visible when the demo runs.
- `}` — Closes `checkFluidLevels()`.
- `void recordInspectionLog() {` — Declares the second concrete invariant step, also package-private for the same reason: it is an internal shared step, not a customization hook. The `{` opens its body.
- `System.out.println("Recording inspection log entry");` — Writes the line `Recording inspection log entry` to standard output. Identical for every vehicle, which is exactly why it lives in the base class instead of being duplicated in each subclass.
- `}` — Closes `recordInspectionLog()`.
- `}` — Closes the `AbstractInspectionProcedure` class body. This single class contains the whole pattern: a fixed skeleton (`performInspection`), two shared steps, and two deferred steps.

`CarInspectionProcedure.java`

```
package com.design.patterns.templatemethod.template.concrete;

import com.design.patterns.templatemethod.template.AbstractInspectionProcedure;

public class CarInspectionProcedure extends AbstractInspectionProcedure {

    @Override
    public void checkSafetySystems() {
        System.out.println("Testing airbags, seatbelts and ABS");
    }

    @Override
    public void issueCertificate() {
        System.out.println("Issuing four-wheeler roadworthiness certificate");
    }
}
```

Line by line:

- `package com.design.patterns.templatemethod.template.concrete;` — Declares that this class lives in the `template.concrete` sub-package — the home for concrete inspection procedures, kept one level below the abstract `template` package that holds the skeleton.
- `import com.design.patterns.templatemethod.template.AbstractInspectionProcedure;` — Brings the base class `AbstractInspectionProcedure` from the parent `template` package into scope so it can be named by its simple name (`AbstractInspectionProcedure`) below. Without this import the fully-qualified name would be required on the `extends` clause, because the base class lives in a different package.
- `public class CarInspectionProcedure extends AbstractInspectionProcedure {` — Declares a `public`, non-abstract class `CarInspectionProcedure` that extends `AbstractInspectionProcedure`. Being non-abstract, it *must* provide bodies for both inherited `abstract` methods, or it would not compile. By extending `AbstractInspectionProcedure` it inherits the `final` template method and the two concrete steps unchanged. The `{` opens the class body.
- `@Override` — An annotation asserting that the method beneath it overrides an inherited method. It is not required for correctness but is a safety net: if the signature did not actually match an inherited `abstract` method (a typo like `chekSafetySystems()`), the compiler would reject it, catching the mistake immediately.
- `public void checkSafetySystems() {` — Provides `CarInspectionProcedure`'s implementation of the `abstract` `checkSafetySystems()` step. The signature (`public void checkSafetySystems()`) matches the base declaration exactly, which is what makes it a valid override. The `{` opens the body.
- `System.out.println("Testing airbags, seatbelts and ABS");` — The car-specific safety-check behavior: prints `Testing airbags, seatbelts and ABS`. This is the line the template method reaches when it calls `checkSafetySystems()` on a `CarInspectionProcedure` object.
- `}` — Closes `checkSafetySystems()`.

- `@Override` — Again asserts the next method overrides an inherited abstract method, with the same compile-time safety guarantee.
- `public void issueCertificate() {` — Provides `CarInspectionProcedure`'s implementation of the `abstract issueCertificate()` step; the signature matches the base declaration. The `{` opens the body.
- `System.out.println("Issuing four-wheeler roadworthiness certificate");` — The car-specific certificate behavior: prints `Issuing four-wheeler roadworthiness certificate`.
- `}` — Closes `issueCertificate()`.
- `}` — Closes the `CarInspectionProcedure` class. Note what is *absent*: there is no `performInspection()`, no `checkFluidLevels()`, no `recordInspectionLog()`. The subclass supplies only the two steps that vary and inherits everything else. That inversion — the subclass fills in steps rather than driving the algorithm — is the whole point of the pattern.

MotorcycleInspectionProcedure.java

```
package com.design.patterns.templateMethod.template.concrete;

import com.design.patterns.templateMethod.template.AbstractInspectionProcedure;

public class MotorcycleInspectionProcedure extends AbstractInspectionProcedure {

    @Override
    public void checkSafetySystems() {
        System.out.println("Testing brake lights and horn");
    }

    @Override
    public void issueCertificate() {
        System.out.println("Issuing two-wheeler roadworthiness certificate");
    }
}
```

Line by line:

- `package com.design.patterns.templateMethod.template.concrete;` — Same sub-package as `CarInspectionProcedure`; both concrete procedures sit together under `template.concrete`.
- `import com.design.patterns.templateMethod.template.AbstractInspectionProcedure;` — Imports the base class so it can be named simply as `AbstractInspectionProcedure` in the `extends` clause below.
- `public class MotorcycleInspectionProcedure extends AbstractInspectionProcedure {` — Declares the second concrete inspection procedure, again extending `AbstractInspectionProcedure` and thus obligated to implement both abstract steps. The `{` opens the class body.
- `@Override` — Marks the following method as an override of an inherited abstract method, with compile-time checking.
- `public void checkSafetySystems() {` — Provides `MotorcycleInspectionProcedure`'s version of the `checkSafetySystems()` step, signature matching the base declaration. The `{` opens the body.
- `System.out.println("Testing brake lights and horn");` — The motorcycle-specific safety-check behavior: prints `Testing brake lights and horn`. This is what the template method reaches when `checkSafetySystems()` is dispatched on a `MotorcycleInspectionProcedure`.
- `}` — Closes `checkSafetySystems()`.
- `@Override` — Marks the next override, same safety guarantee.
- `public void issueCertificate() {` — Provides `MotorcycleInspectionProcedure`'s version of `issueCertificate()`, signature matching the base. The `{` opens the body.
- `System.out.println("Issuing two-wheeler roadworthiness certificate");` — The motorcycle-specific certificate behavior: prints `Issuing two-wheeler roadworthiness certificate`.
- `}` — Closes `issueCertificate()`.
- `}` — Closes the `MotorcycleInspectionProcedure` class. Like `CarInspectionProcedure`, it overrides only the two varying steps and inherits the identical skeleton — proving the algorithm is defined once and reused, not copied.

VehicleInspectionStation.java

```
package com.design.patterns.templateMethod;

import com.design.patterns.templateMethod.template.AbstractInspectionProcedure;
```

```

import com.design.patterns.templatemethod.template.concrete.CarInspectionProcedure;
import com.design.patterns.templatemethod.template.concrete.MotorcycleInspectionProcedure;

public class VehicleInspectionStation {

    public static void main(String[] args) {
        System.out.println("Template Method Design Pattern");

        System.out.println("----Time to inspect the car.");
        AbstractInspectionProcedure carInspection = new CarInspectionProcedure();
        carInspection.performInspection();
        System.out.println("----Now time to inspect the motorcycle.");

        AbstractInspectionProcedure motorcycleInspection = new MotorcycleInspectionProcedure();
        motorcycleInspection.performInspection();
    }
}

```

Line by line:

- `package com.design.patterns.templatemethod;` — Declares that the driver class lives in the top-level `...templatemethod` package, one level above `template`. Keeping the demo in the parent package, with the pattern's classes in sub-packages beneath it, separates the client from the pattern implementation.
- `import com.design.patterns.templatemethod.template.AbstractInspectionProcedure;` — Imports the base type so the two local variables below can be declared as `AbstractInspectionProcedure`. Referencing the objects through the base type (not the concrete type) is deliberate — it shows the client depends only on the abstraction.
- `import com.design.patterns.templatemethod.template.concrete.CarInspectionProcedure;` — Imports the concrete `CarInspectionProcedure` from the `template.concrete` sub-package so it can be constructed by its simple name.
- `import com.design.patterns.templatemethod.template.concrete.MotorcycleInspectionProcedure;` — Imports the concrete `MotorcycleInspectionProcedure` for the same reason.
- `public class VehicleInspectionStation {` — Declares the `public` driver class. `public` is required so the JVM can resolve the `main` entry point by name when the class is named on the command line. This class is purely the client — it does not participate in the pattern's structure. The `{` opens the class body.
- `public static void main(String[] args) {` — The standard Java entry point. `public` so the JVM launcher can call it; `static` so it runs without constructing a `VehicleInspectionStation` instance; `void` because it returns nothing; `String[] args` receives command-line arguments (none are used here). The `{` opens the method body.
- `System.out.println("Template Method Design Pattern");` — Prints a title banner to standard output so the demo's output is self-labeling.
- `System.out.println("----Time to inspect the car.");` — Prints a section header announcing the car run, so the two inspections are visually separated in the output.
- `AbstractInspectionProcedure carInspection = new CarInspectionProcedure();` — Constructs a `CarInspectionProcedure` object but stores it in a variable typed as the base class `AbstractInspectionProcedure`. This is *programming to the abstraction*: the client holds the general type and will call only methods declared on it. The concrete class name appears only here, at construction.
- `carInspection.performInspection();` — Invokes the `final` template method on the car object. This single call runs the whole fixed algorithm: `checkFluidLevels()` (base), then `checkSafetySystems()` (dispatched to `CarInspectionProcedure.checkSafetySystems()`), then `recordInspectionLog()` (base), then `issueCertificate()` (dispatched to `CarInspectionProcedure.issueCertificate()`). The client never calls `checkSafetySystems()` itself — the template calls it. This produces four output lines for the car.
- `System.out.println("----Now time to inspect the motorcycle.");` — Prints the header for the motorcycle run. (Note it is printed *after* the car inspection is finished, since the previous line ran to completion before this executes.)
- `AbstractInspectionProcedure motorcycleInspection = new MotorcycleInspectionProcedure();` — Constructs a `MotorcycleInspectionProcedure`, again held through the base type `AbstractInspectionProcedure`. Same coding-to-abstraction style as the car variable.
- `motorcycleInspection.performInspection();` — Invokes the *same* template method on the motorcycle object. It runs the identical skeleton, but `checkSafetySystems()` and `issueCertificate()` now dispatch to `MotorcycleInspectionProcedure`'s overrides, producing motorcycle-specific lines while the shared steps print exactly as they did for the car. This demonstrates the core payoff: one algorithm, two results, chosen by polymorphism.
- `}` — Closes `main()`; with nothing left to run, the JVM exits.
- `}` — Closes the `VehicleInspectionStation` class body.

Why these design decisions

Why put the algorithm in the base class instead of in each subclass? Because the *order and structure* are the part that must not vary. Every vehicle follows the same four-step inspection routine in the same sequence. If each subclass wrote its own `performInspection()`, that shared sequence would be duplicated across all of them, and any subclass could accidentally reorder steps (issue the certificate before recording the log), skip one, or add a fifth. Centralizing the skeleton means the routine is defined once and reused — the pattern's core benefit is *reuse of the algorithm's structure*.

Why is the template method `final`? `final` forbids subclasses from overriding `performInspection()`. That is deliberate: the base class is saying "you may customize the *steps*, but you may not rewrite the *routine*." Without `final`, a subclass could override the template, change the order, or drop `checkFluidLevels()` entirely — defeating the very invariant the pattern exists to protect. Making it `final` locks the algorithm's shape while still allowing per-step variation through the abstract methods.

Why are `checkSafetySystems()` and `issueCertificate()` `abstract`? Because they are the steps that genuinely differ and for which the base class has no sensible default. Declaring them `abstract` does two things: (1) it *forces* every concrete subclass to supply them — you cannot compile an `AbstractInspectionProcedure` subclass that forgets `checkSafetySystems()`, so the compiler enforces the contract; and (2) it *documents* precisely which two steps are the intended customization points.

Why are `checkFluidLevels()` and `recordInspectionLog()` `concrete (and not abstract)`? Because they are invariant — identical for every vehicle. Implementing them once in the base class shares the code and signals "these are not customization points." They are package-private rather than `public` because their only caller is the template method inside the same package; exposing them publicly would invite subclasses to treat them as override hooks, which they are not.

Why does the client hold the objects as `AbstractInspectionProcedure` rather than `CarInspectionProcedure` / `MotorcycleInspectionProcedure`? To depend only on the abstraction. The client calls the same method, `performInspection()`, on every vehicle; the differences are resolved by polymorphism at runtime. Coding to the base type means the client would not change if a third vehicle class were added.

Why is this "inversion of control" (the Hollywood Principle)? Notice who calls whom. The subclass does *not* call the base class to run the algorithm; instead the base class's template method calls *down* into the subclass's overridden steps at the right moments — "don't call us, we'll call you." The high-level flow stays in the base class; the low-level details are supplied by subclasses and invoked *by* the framework method. That inversion is exactly what lets the base class guarantee the order.

Template Method vs. Strategy. Both let behavior vary, but differently. Template Method varies *individual steps* via **inheritance** — subclasses override abstract methods and the skeleton is fixed at compile time in the base class. Strategy varies the *whole algorithm* via **composition** — you inject a different strategy object at runtime. Reach for Template Method when the overall sequence is fixed and only specific steps change; reach for Strategy when the entire algorithm should be swappable at runtime.

Execution flow (step-by-step trace of what happens when `main()` runs)

1. The JVM loads `VehicleInspectionStation` and begins executing `main()`.
2. `System.out.println("Template Method Design Pattern")` prints the title line.
3. `System.out.println("----Time to inspect the car.")` prints the car header.
4. `new CarInspectionProcedure()` constructs a car-inspection object; it is assigned to an `AbstractInspectionProcedure`-typed variable `carInspection`.
5. `carInspection.performInspection()` is called. Because `performInspection()` is `final`, the version in `AbstractInspectionProcedure` runs; there is no override to dispatch to.
6. Inside `performInspection()`, `checkFluidLevels()` runs (base class) and prints `Checking engine oil and coolant levels`.
7. `checkSafetySystems()` is called; dynamic dispatch routes it to `CarInspectionProcedure.checkSafetySystems()`, which prints `Testing airbags, seatbelts and ABS`.
8. `recordInspectionLog()` runs (base class) and prints `Recording inspection log entry`.
9. `issueCertificate()` is called; dispatch routes it to `CarInspectionProcedure.issueCertificate()`, which prints `Issuing four-wheeler roadworthiness certificate`. The car run is complete.
10. `System.out.println("----Now time to inspect the motorcycle.")` prints the motorcycle header.
11. `new MotorcycleInspectionProcedure()` constructs a motorcycle-inspection object, assigned to `AbstractInspectionProcedure`-typed variable `motorcycleInspection`.
12. `motorcycleInspection.performInspection()` is called — the *same* `final` template method.
13. `checkFluidLevels()` runs (base class) and prints `Checking engine oil and coolant levels` — identical to the car run.
14. `checkSafetySystems()` is dispatched to `MotorcycleInspectionProcedure.checkSafetySystems()`, which prints `Testing brake lights and horn`.
15. `recordInspectionLog()` runs (base class) and prints `Recording inspection log entry` — again identical to the car run.
16. `issueCertificate()` is dispatched to `MotorcycleInspectionProcedure.issueCertificate()`, which prints `Issuing two-wheeler roadworthiness certificate`. The motorcycle run is complete.
17. `main()` returns; the JVM exits.

The shared steps (lines from `checkFluidLevels` and `recordInspectionLog`) print identically in both runs, while the safety-check and certificate lines differ — and the order is the same both times. That is Template Method working exactly as intended.

Expected output

```
Template Method Design Pattern
-----Time to inspect the car.
Checking engine oil and coolant levels
Testing airbags, seatbelts and ABS
Recording inspection log entry
Issuing four-wheeler roadworthiness certificate
-----Now time to inspect the motorcycle.
Checking engine oil and coolant levels
Testing brake lights and horn
Recording inspection log entry
Issuing two-wheeler roadworthiness certificate
```

How to run

```
# From the module root directory
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile

java -cp target/classes com.design.patterns.templatemethod.VehicleInspectionStation
```

The `-o` (offline) flag works once the parent reactor and dependencies are already in your local Maven cache; drop it on a first build if resolution fails. The demo is a plain `main` — the pattern is about class structure and needs no Spring container to run.